ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 9:

Aim

To implement and solve the Traveling Salesman Problem (TSP) using a backtracking search algorithm in Python to find the minimum-cost tour that visits every city exactly once and returns to the starting city.

Algorithm

1. Define Problem Structure: Represent cities and distances using a cost matrix or adjacency matrix where entry (i, j) represents the travel cost from city i to city j.
2. Initialize Tracking Variables: Set the starting city (e.g., city 0), mark it as visited, and initialize variables to track the minimum cost found so far and the best tour path.
3. Backtracking Search (Recursive DFS):
    - Check if all cities have been visited. If so, add the cost to return from the current city back to the starting city.
    - If this total tour cost is less than the current minimum cost, update the minimum cost and store the current path.
4. Branching and Pruning:
    - Iterate through all unvisited neighboring cities.
    - If moving to a neighboring city incurs a partial cost already exceeding the current minimum cost, prune that branch.
    - Otherwise, recursively explore that city, updating the path and current cost.
5. Backtrack: Unmark the city as visited (backtrack) to explore alternative routes and output the optimal tour and minimum cost.

Code

```
class TravelingSalesman:
    def __init__(self, cost_matrix):
        self.graph = cost_matrix
        self.num_cities = len(cost_matrix)
        self.min_cost = float('inf')
        self.best_tour = []

    def tsp_backtrack(self, current_city, count, current_cost, visited, tour):
        if count == self.num_cities:
            total_cost = current_cost + self.graph[current_city][tour[0]]
            if total_cost < self.min_cost:
                self.min_cost = total_cost
                self.best_tour = list(tour) + [tour[0]]
            return

        for next_city in range(self.num_cities):
            if not visited[next_city] and self.graph[current_city][next_city] > 0:
                if current_cost + self.graph[current_city][next_city] < self.min_cost:
                    visited[next_city] = True
                    tour.append(next_city)

                    self.tsp_backtrack(next_city, count + 1, current_cost + self.graph[current_city][next_city], visited, tour)

                    tour.pop()
                    visited[next_city] = False

    def solve(self, start_city=0):
        visited = [False] * self.num_cities
        visited[start_city] = True
        tour = [start_city]

        self.tsp_backtrack(start_city, 1, 0, visited, tour)
        return self.best_tour, self.min_cost

# Example distance matrix for 4 cities
cost_matrix = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

tsp = TravelingSalesman(cost_matrix)
optimal_tour, minimum_cost = tsp.solve(start_city=0)

print("Optimal Tour:", " -> ".join(map(str, optimal_tour)))
print(f"Minimum Tour Cost: {minimum_cost}")
```

```
============= RESTART: C:/Users/Admin/OneDrive/Documents/aiexp9.p
Optimal Tour: 0 -> 1 -> 3 -> 2 -> 0
Minimum Tour Cost: 80
```

Result

The Traveling Salesman program successfully evaluated all possible permutations of city sequences using backtracking. Starting from city 0, the algorithm calculated the total travel weights for valid paths, pruned suboptimal branches, and identified the absolute minimum-cost Hamiltonian cycle.

Conclusion

The program successfully demonstrated how to solve the NP-hard Traveling Salesman Problem using systematic state-space tree exploration and backtracking. By tracking minimum costs dynamically and pruning branches exceeding known upper bounds, the algorithm efficiently determines the optimal closed tour visiting every city with minimal total distance.